

TT File Processor - Technical Report

Project: Tunisie Telecom File Processor

Application type: Python web application wrapped as a Windows desktop application

Repository: `koussay0/Files-Organization-System-TT`

1. Project Purpose

TT File Processor is a data-processing tool for importing, cleaning, transforming, comparing, merging, and exporting structured files. It provides a browser-style dashboard and can also run as a Windows desktop application in its own window.

The application is rule-based. It does not require machine learning for its current features because the operations are deterministic data-processing tasks.

2. Main Features

File input

The application can load:

- CSV files
- TXT files
- XLS files
- XLSX files
- PDF files containing tables or delimited text

Text files use automatic delimiter detection for comma, semicolon, tab, and pipe-separated data. Uploaded files are normalized to CSV for later single-file operations.

Single-file processing

The single-file workflow supports:

- Uploading a file and displaying a preview
- Displaying row count, column count, and column names
- Sorting by one column
- Sorting by the complete row
- Detecting duplicate rows
- Detecting duplicates using selected columns
- Removing duplicates
- Detecting consecutive numeric sequences
- Splitting a file by number of output files
- Splitting a file by rows per output file

- Downloading split files as a ZIP archive
- Generating formatted values from a reusable template
- Injecting text before or after column values
- Removing text from column values
- Applying addition, subtraction, multiplication, or division to numeric columns
- Reordering columns
- Adding a column
- Removing a column
- Extracting selected columns

Two-file processing

The application supports:

- Comparing two files by their complete rows
- Comparing two files by a selected column
- Displaying rows present in both files
- Displaying rows only in the first file
- Displaying rows only in the second file
- Merging selected columns from two files side by side
- Exporting comparison and merge results

The merge operation is a side-by-side concatenation based on row position. It is not a database-style key join.

Export

Results can be exported as:

- CSV
- TXT
- XLSX/Excel
- PDF

CSV and TXT exports support custom separators. PDF output is rendered as a table and is limited to a configurable number of rows for practical performance.

User interface

The interface includes:

- Flask/Jinja2 templates
- Bootstrap 5 layout components
- Custom CSS with organized panels and color accents
- English and French language options
- Session-based language selection

- Dashboard navigation for single-file, comparison, and merge workflows
- Preview tables and flash messages for errors and status information

3. Technologies Used

Technology | Role

Python | Main programming language

Flask | Web framework, routing, sessions, application factory

Jinja2 | Server-rendered HTML templates; included with Flask

Pandas | DataFrame storage and data-processing engine

Bootstrap 5 | Responsive UI layout and base components

Custom CSS | Application theme, colors, spacing, cards, and dashboard styling

OpenPyXL | Reading and writing XLSX Excel files

xlrd | Support for legacy XLS files

pdfplumber | Reading tables and text from PDF files

ReportLab | Generating PDF exports

python-dotenv | Environment configuration support

pytest | Automated unit and integration-style tests

pywebview | Embedding the local Flask interface in a native desktop window

PyInstaller | Packaging the Python application into a Windows executable

The exact runtime dependencies are listed in `requirements.txt`.

4. How the Web App Becomes a Windows App

The project uses a hybrid desktop architecture rather than rewriting the interface as a separate native GUI.

Startup sequence

1. `run.py` creates the Flask application with `create\_app()`.
2. Flask starts in a background thread on `http://127.0.0.1:5000`.
3. `pywebview.create\_window()` opens that local address in a desktop window.
4. `webview.start()` runs the desktop window event loop.
5. The user sees the Flask/Jinja2 application as a Windows desktop application.

Framework responsible for the window

The main framework used to create the Windows window is pywebview. It provides a native window containing the local web interface.

Flask remains responsible for the application server and routes. Pywebview is the desktop wrapper. PyInstaller is used afterward to package the Python code, templates, static files, and dependencies into an `.exe` distribution.

This approach provides:

- Reuse of the existing web UI
- No need to rebuild the interface in Tkinter, PyQt, or .NET
- A familiar desktop window for Windows users
- A relatively small change to the existing Flask architecture

5. Application Architecture

```text

run.py

+-- pywebview desktop window

+-- **local Flask server: 127.0.0.1:5000**

+-- app/\_\_init\_\_.py

+-- routes/main.py

+-- routes/single\_file.py

+-- routes/multi\_file.py

+-- services/file\_loader.py

+-- services/operations.py

+-- services/exporters.py

+-- templates/ and static/

### Application factory: `app/\_\_init\_\_.py`

The application factory:

- Creates the Flask application
- Configures upload and output folders
- Creates required directories
- Registers blueprints
- Provides the favicon route
- Provides language/session context for English and French templates

### Main routes: `app/routes/main.py`

The main blueprint provides:

- Dashboard/home page
- Language selection route

### Single-file routes: `app/routes/single\_file.py`

This blueprint handles upload, preview, sorting, duplicate operations, sequence detection, splitting, template

### Multi-file routes: `app/routes/multi\_file.py`

This blueprint handles two-file comparison, comparison-result export, and side-by-side merging.

### File loader: `app/services/file\_loader.py`

The loader centralizes input parsing. It determines the extension, detects text delimiters, reads Excel files,

### Operations service: `app/services/operations.py`

This module contains pure Pandas-oriented functions. Keeping the data logic separate from Flask routes makes t

```
Export service: `app/services/exporters.py`
```

This module centralizes output generation for CSV, TXT, Excel, and PDF. Routes call the shared exporter instead.

## ## 6. Data and Session Management

Uploaded and generated files are stored under:

- `instance/uploads/`
- `instance/outputs/`

The current workflow stores generated filenames in the Flask session. Single-file operations use a normalized filename.

This is suitable for a local desktop application or small internal deployment. A multi-user server deployment would require a database.

## ## 7. Testing

The project includes an expanded test suite in `tests/test\_operations.py`. It covers:

- Sorting
- Duplicate detection and removal
- Sequence detection
- File splitting
- Template generation
- String and math transformations
- Column management
- File comparison
- File merging
- File extension and delimiter detection
- Basic statistics
- CSV, TXT, Excel, and PDF export
- End-to-end operation combinations
- Empty data, null values, and special characters
- Expected error cases

The test suite was verified with:

```
```bash
.venv/Scripts/python.exe -m pytest tests/test_operations.py -v --tb=short
```

Verified result: 67 tests passed.

8. Current Limitations and Recommended Improvements

The current implementation is functional, but the following improvements would strengthen it for production use:

1. Add validation for invalid column names, invalid split values, and incompatible file schemas before Pandas operations run.
2. Add automated Flask route tests for upload, language switching, export downloads, and error redirects.
3. Add file cleanup for old uploads and generated outputs.
4. Add progress feedback and chunked processing for very large files.
5. Replace the development secret key with an environment variable in deployed builds.
6. Add CSRF protection if the application is deployed beyond a trusted local environment.
7. Add batch folder processing and saved processing presets if repeated workflows are required.
8. Improve accessibility with explicit labels, keyboard navigation, and screen-reader-friendly status messages.
9. Validate the packaged executable on a clean Windows machine with no development environment installed.
10. Keep the README synchronized with the current implementation and desktop packaging instructions.

9. Machine Learning Assessment

Machine learning is not required for the current application. Sorting, duplicate removal, sequence detection, transformations, file comparison, merging, and exporting all have clear deterministic rules.

Machine learning could be added later for optional capabilities such as:

- Automatic anomaly detection
- Classification of unknown columns
- Data-cleaning recommendations
- Duplicate matching when records are similar but not identical
- Predictive quality scoring

Those would be extensions, not requirements for the current application.

10. Summary

TT File Processor is a Flask and Pandas data-processing application with a Bootstrap/Jinja2 interface. It handles common structured-file workflows from upload through transformation and export. The Windows desktop experience is provided by pywebview, which displays the local Flask application inside a native window, while PyInstaller packages the application into an executable.

The architecture separates routes, data operations, file loading, exporting, templates, and static styling. This makes the project understandable, testable, and suitable for further feature development.